Relatório de Desenvolvimento - Fase 2

Desenvolvimento de Software	INF01120
Davi Santos	00599530
Gabriel Schons	00342020
Gabriel Copatti	00342808
Julio Augusto de Castilhos Borges	00590537

Lista de Requisitos Funcionais

O sistema deve:

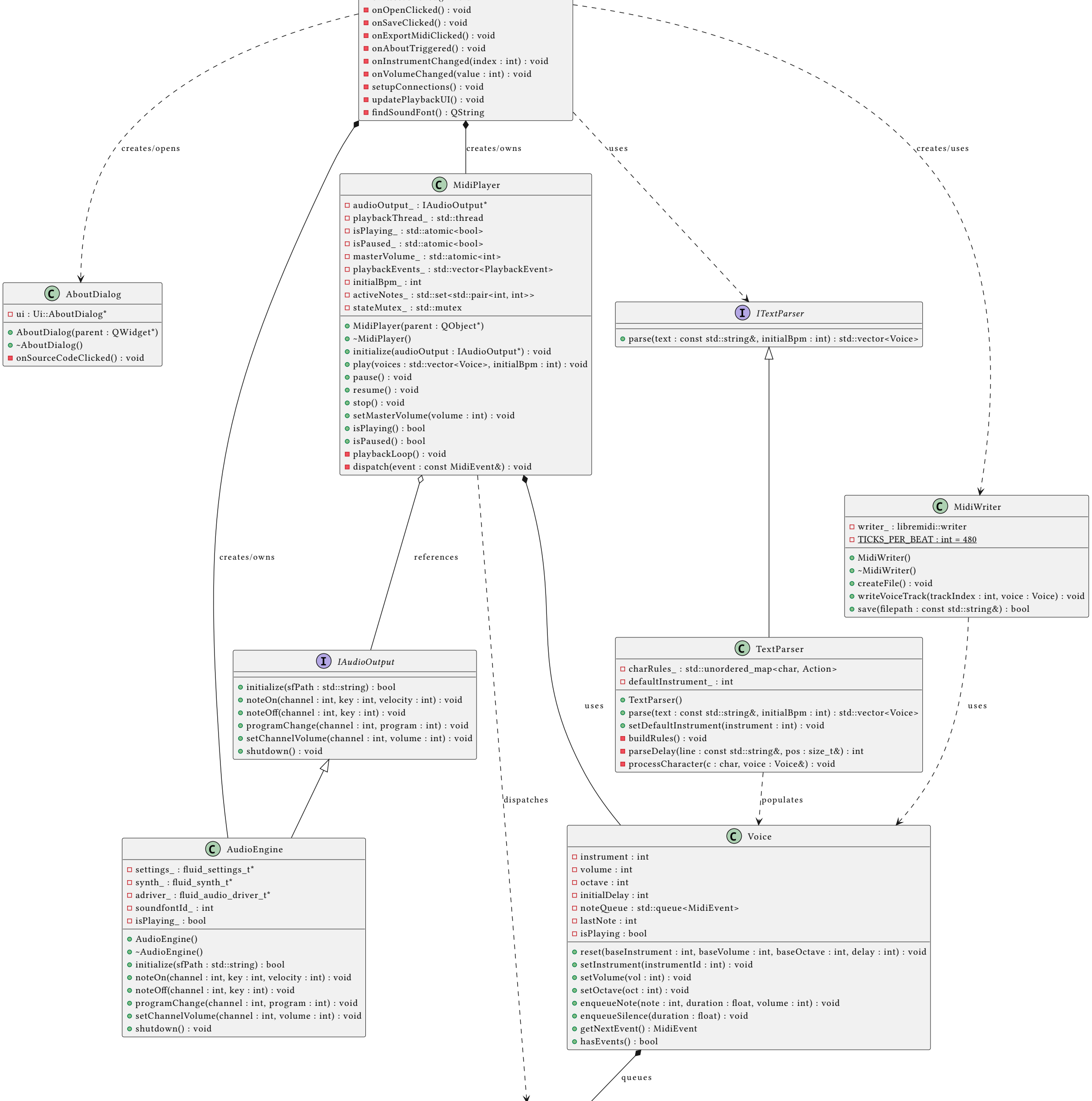
1. Fornecer uma área de edição de texto na interface gráfica para que o usuário possa visualizar, editar e inserir sequências de texto livremente.
2. Permitir carregar sequências de texto a partir de arquivos no formato texto simples (`.txt`) e salvar o texto editado de volta no mesmo formato.
3. Tratar cada linha do texto de entrada como uma voz musical independente ($V_0, V_1, V_2, \ldots$), processando-as de forma polifônica e simultânea.
4. Mapear caracteres específicos do texto para eventos musicais locais (notas, volumes locais, instrumentos locais e alterações locais de oitava) de acordo com regras estritas de interpretação.
5. Suportar atrasos iniciais individuais para cada voz através da sintaxe `[n]` inserida no início da linha de texto correspondente, que introduz n tempos de silêncio antes do processamento.
6. Processar comandos de alteração de andamento (`>` e `<`) de escopo global, alterando o BPM do sistema para todas as vozes em tempo real durante a reprodução.
7. Exportar a composição gerada para arquivos de música no formato MIDI padrão (`.mid`), organizando cada voz em trilhas MIDI independentes.
8. Sintetizar e reproduzir o áudio resultante em tempo real utilizando a biblioteca FluidSynth com carregamento de tabelas de ondas do formato SoundFont.
9. Executar a síntese de som de maneira assíncrona em uma thread dedicada separada, impedindo o bloqueio ou congelamento da interface gráfica durante o processamento de áudio.
10. Permitir controlar a execução da música através de comandos de Tocar, Pausar e Parar, além de possibilitar o ajuste de parâmetros globais iniciais na interface.

Requisitos Não Funcionais

1. O sistema deve seguir rigorosamente os princípios de programação orientada a objetos (SOLID).
2. O código-fonte principal deve ser escrito em C++ com suporte ao padrão C++26.
3. A interface gráfica deve ser construída utilizando a biblioteca Qt6 e arquivos de definição visual do Qt Designer (`.ui`).
4. A síntese sonora em tempo real deve ser realizada através do sintetizador FluidSynth integrado ao código.
5. O ciclo de vida do build da aplicação deve ser configurado via CMake, gerenciando dependências do Qt6 e FluidSynth de maneira reprodutível.
6. O controle de versões do projeto deve ser gerenciado através do sistema Git.
7. Testes unitários de validação da geração de eventos musicais devem ser implementados com suporte do CTest.

Definição das Classes

Diagrama UML


MainWindow
ui : Ui::MusicMachine*
audioEngine : std::unique_ptr<AudioEngine>
midiPlayer : std::unique_ptr<MidiPlayer>
soundfontPath : QString
MainWindow(parent : QWidget*)
~MainWindow()
onPlayClicked() : void
onResetClicked() : void
onOpenClicked() : void
onSaveClicked() : void
onExportMidiClicked() : void
onAboutTriggered() : void
onInstrumentChanged(index : int) : void
onVolumeChanged(value : int) : void
setupConnections() : void
updatePlaybackUI() : void
findSoundFont() : QString
creates/opens
creates/owns
uses
creates/uses
AboutDialog
ui : Ui::AboutDialog*
AboutDialog(parent : QWidget*)
~AboutDialog()
onSourceCodeClicked() : void
MidiPlayer
audioOutput_ : IAudioOutput*
playbackThread_ : std::thread
isPlaying_ : std::atomic<bool>
isPaused_ : std::atomic<bool>
masterVolume_ : std::atomic<int>
playbackEvents_ : std::vector<PlaybackEvent>
initialBpm_ : int
activeNotes_ : std::set<std::pair<int, int>>
stateMutex_ : std::mutex
MidiPlayer(parent : QObject*)
~MidiPlayer()
initialize(audioOutput : IAudioOutput*) : void
play(voices : std::vector<Voice>, initialBpm : int) : void
pause() : void
resume() : void
stop() : void
setMasterVolume(volume : int) : void
isPlaying() : bool
isPaused() : bool
playbackLoop() : void
dispatch(event : const MidiEvent&) : void
ITextParser
parse(text : const std::string&, initialBpm : int) : std::vector<Voice>
MidiWriter
writer_ : libremidi::writer
TICKS_PER_BEAT : int = 480
MidiWriter()
~MidiWriter()
createFile() : void
writeVoiceTrack(trackIndex : int, voice : Voice) : void
save(filepath : const std::string&) : bool
creates/owns
references
IAudioOutput
initialize(sfPath : std::string) : bool
noteOn(channel : int, key : int, velocity : int) : void
noteOff(channel : int, key : int) : void
programChange(channel : int, program : int) : void
setChannelVolume(channel : int, volume : int) : void
shutdown() : void
dispatches
uses
uses
TextParser
charRules_ : std::unordered_map<char, Action>
defaultInstrument_ : int
TextParser()
parse(text : const std::string&, initialBpm : int) : std::vector<Voice>
setDefaultInstrument(instrument : int) : void
buildRules() : void
parseDelay(line : const std::string&, pos : size_t&) : int
processCharacter(c : char, voice : Voice&) : void
populates
AudioEngine
settings_ : fluid_settings_t*
synth_ : fluid_synth_t*
adriver_ : fluid_audio_driver_t*
soundfontId_ : int
isPlaying_ : bool
AudioEngine()
~AudioEngine()
initialize(sfPath : std::string) : bool
noteOn(channel : int, key : int, velocity : int) : void
noteOff(channel : int, key : int) : void
programChange(channel : int, program : int) : void
setChannelVolume(channel : int, volume : int) : void
shutdown() : void
Voice
instrument : int
volume : int
octave : int
initialDelay : int
noteQueue : std::queue<MidiEvent>
lastNote : int
isPlaying : bool
reset(baseInstrument : int, baseVolume : int, baseOctave : int, delay : int) : void
setInstrument(instrumentId : int) : void
setVolume(vol : int) : void
setOctave(oct : int) : void
enqueueNote(note : int, duration : float, volume : int) : void
enqueueSilence(duration : float) : void
getNextEvent() : MidiEvent
hasEvents() : bool
queues
MidiEvent
type : MidiEventType
pitch : int
velocity : int
timestamp : double
duration : double
channel : int
value : int
MidiEventType
NoteOn
NoteOff
ProgramChange
VolumeChange
BpmChange

Classe Voice

Atributos:

- `instrument: int` - Armazena o ID do instrumento MIDI ativo para a trilha (0-127)
- `volume: int` - Armazena o volume atual da trilha (0-127)
- `octave: int` - Armazena a oitava local atual (0-9)
- `initialDelay: int` - Armazena a quantidade de batidas de atraso inicial (`[n]`) da voz
- `noteQueue: std::queue<MidiEvent>` - Fila de eventos musicais (notas e silêncios) agendados
- `lastNote: int` - Armazena a última nota MIDI reproduzida (para regras de repetição)
- `isPlaying: bool` - Indica se a voz está sendo ativamente processada

Métodos:

- `reset(baseInstrument: int, baseVolume: int, baseOctave: int, delay: int): void` - Restaura o estado interno e parâmetros iniciais da voz
- `setInstrument(instrumentId: int): void` - Modifica o instrumento da voz
- `setVolume(vol: int): void` - Define o volume local
- `setOctave(oct: int): void` - Define a oitava da voz, com limites inferior (0) e superior (9)
- `enqueueNote(note: int, duration: float, volume: int): void` - Adiciona um evento de nota MIDI à fila
- `enqueueSilence(duration: float): void` - Adiciona um evento de pausa à fila de eventos
- `getNextEvent(): MidiEvent` - Retorna e remove o próximo evento MIDI da fila
- `hasEvents(): bool` - Verifica se há eventos pendentes na fila de reprodução

Classe TokenProcessor

Atributos:

- `mappingRules: std::unordered_map<char, Instruction>` - Dicionário que mapeia caracteres para instruções musicais correspondentes

Métodos:

- `tokenize(input: string, voices: std::vector<Voice>&): void` - Processa o texto completo de entrada, particionando em linhas e configurando a fila de notas de cada voz
- `parseLine(line: string, voiceIndex: int, voice: Voice&): void` - Analisa lexicalmente uma linha individual de texto, gerando e enfileirando eventos de música na voz correspondente
- `parseDelay(line: string): int` - Analisa o prefixo de atraso (`[n]`) no início da string, retornando o número de tempos de atraso extraído

Classe MusicMachine

Atributos:

- `voices: std::vector<Voice>` - Vetor contendo a lista de vozes polifônicas gerenciadas
- `globalBPM: int` - Armazena o andamento global em batidas por minuto (BPM)
- `tokenProcessor: TokenProcessor` - Referência para a classe de interpretação de texto
- `audioEngine: AudioEngine` - Referência para a classe gerenciadora do motor de síntese

Métodos:

- `initialize(text: string): void` - Prepara a máquina para reprodução realizando o parsing do texto e inicializando as vozes
- `play(): void` - Inicia a reprodução polifônica concorrente de todas as vozes
- `pause(): void` - Pausa temporariamente a execução global
- `stop(): void` - Finaliza a reprodução e reseta o cursor de eventos
- `setBPM(bpm: int): void` - Define o BPM global inicial
- `adjustBPM(delta: int): void` - Ajusta o BPM global dinamicamente durante a reprodução
- `exportToMidi(filepath: string): bool` - Exporta a sequência de eventos de todas as vozes para um arquivo padrão (`.mid`)

Classe AudioEngine

Atributos:

- `settings: fluid_settings_t*` - Configurações internas de alocação do sintetizador FluidSynth
- `synth: fluid_synth_t*` - Instância ativa do gerador de síntese FluidSynth
- `adriver: fluid_audio_driver_t*` - Driver de saída de áudio de tempo real do sistema operacional
- `soundfontId: int` - ID identificador do arquivo de SoundFont (`.sf2`) carregado
- `isPlaying: bool` - Estado atual da síntese sonora

Métodos:

- `initialize(sfPath: string): bool` - Configura o motor, carrega a SoundFont e ativa o driver de som
- `noteOn(channel: int, key: int, velocity: int): void` - Envia instrução MIDI para tocar nota no sintetizador
- `noteOff(channel: int, key: int): void` - Envia instrução MIDI para silenciar nota
- `programChange(channel: int, program: int): void` - Associa um instrumento MIDI ao canal informado
- `setChannelVolume(channel: int, volume: int): void` - Ajusta o volume do canal de síntese específico
- `shutdown(): void` - Para a síntese e desaloca recursos de memória de áudio

Classe MidiWriter

Atributos:

- `writer_: libremidi::writer` - Instância de controle de escrita de arquivos MIDI polifônicos
- `TICKS_PER_BEAT: int` - Resolução de tempo (constante, padrão 480 ticks/beat)

Métodos:

- `createFile(): void` - Inicializa as estruturas necessárias para o arquivo MIDI multi-track
- `writeVoiceTrack(trackIndex: int, voice: Voice): void` - Converte a fila de eventos de uma voz em eventos MIDI e grava em sua trilha dedicada
- `save(filepath: string): bool` - Salva a estrutura criada em disco no formato binário (`.mid`)

Classe MainWindow

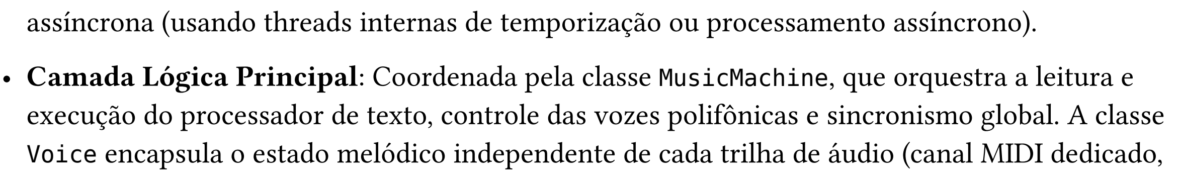
Atributos:

- `musicMachine: MusicMachine` - Instância da lógica central controladora da máquina
- `textEditor: QTextEdit*` - Componente visual da interface gráfica de edição de texto
- `playButton: QPushButton*` - Botão para disparar a execução
- `pauseButton: QPushButton*` - Botão para pausar a execução
- `stopButton: QPushButton*` - Botão para parar e resetar o andamento
- `bpmSpinBox: QSpinBox*` - Seletor numérico para configuração de BPM
- `exportButton: QPushButton*` - Botão para salvar arquivo de música
- `fileMenu: QMenu*` - Menu superior de opções de abertura/gravação de arquivos

Métodos:

- `setupUI(): void` - Liga e conecta dinamicamente os widgets do arquivo (`.ui`) às funções slots de controle
- `onPlayClicked(): void` - Inicializa a reprodução assíncrona
- `onPauseClicked(): void` - Pausa o thread de áudio
- `onStopClicked(): void` - Envia comando de interrupção e reinício de canais
- `onExportClicked(): void` - Solicita caminho de gravação e delega exportação
- `onOpenFile(): void` - Importa arquivos (`.txt`) externos para o painel de texto
- `onSaveFile(): void` - Salva o texto editado pelo painel para disco

GUI em Qt6


File
Playback
Open
Save
100% Volume
Play
Reset
Acoustic Grand Piano
40 BPM

[0] G A H C D Eb V D C H A G
[6] D E F G A H V A G F E D
[12] H C D Eb F G V F Eb D C H

Arquitetura do Sistema

O sistema é estruturado em uma arquitetura modular orientada a objetos projetada para suportar a reprodução e processamento de áudio polifônico em tempo real, sem bloquear ou degradar a responsividade da interface com usuário. O design separa estritamente as seguintes responsabilidades:

- **Camada de Apresentação (GUI)**: Representada pela classe `MainWindow`, consome a interface de usuário definida no arquivo `musicMachine.ui` gerado pelo Qt Designer visando evitar a definição de interfaces de usuário por código-fonte. Esta camada reage a eventos e ações do usuário (carregar/salvar texto, tocar/pausar áudio, configurar parâmetros) e delega as atualizações e disparos para a camada lógica. Toda a manipulação de som e a síntese ocorrem de forma assíncrona (usando threads internas de temporização ou processamento assíncrono).
- **Camada Lógica Principal**: Coordenada pela classe `MusicMachine`, que orquestra a leitura e execução do processador de texto, controle das vozes polifônicas e sincronismo global. A classe `Voice` encapsula o estado melódico independente de cada trilha de áudio (canal MIDI dedicado, oitava local, volume local, instrumento local e fila de eventos), fornecendo uma representação independente dos dados melódicos das trilhas.
- **Camada de Análise Léxica e Semântica**: Implementada pela classe `TokenProcessor`, ela interpreta a entrada textual estruturada em múltiplas linhas. Também interpreta as regras locais (notas A-H, pausas, modulação de oitava/volume/instrumento por caractere) de forma independente para preencher a fila de eventos de cada instância de `Voice`. Ela também reconhece metadados globais (como aceleração de BPM por `>` ou `<`) e inicializações especiais (como o atraso de voz `[n]`).
- **Camada de Síntese e Driver de Áudio**: A classe `AudioEngine` isola a biblioteca externa FluidSynth do restante da lógica da aplicação. Ela realiza a inicialização assíncrona dos buffers, carregamento da SoundFont e despacho de eventos MIDI brutos em canais mapeados para cada voz do sistema. Isso permite que a biblioteca de áudio ou sintetizador físico seja substituído no futuro por outra tecnologia compatível sem que a lógica central da aplicação precise ser reescrita (atendendo ao princípio de Inversão de Dependência).
- **Camada de Exportação**: A classe `MidiWriter` encapsula o empacotamento em formato de trilhas de arquivo MIDI padrão, gravando em disco o resultado estático da interpretação musical.
- **Refinamentos de Implementação**: Durante o desenvolvimento do código-fonte, a classe conceitual `MusicMachine` descrita na modelagem foi refinada em `MidiPlayer` (responsável exclusiva pela gerência da thread assíncrona de reprodução e controle refinado de tempo) e em `MainWindow` (gerenciando a janela gráfica e os slots de interação), garantindo maior coesão de acordo com o Princípio de Responsabilidade Única (SRP). Adicionalmente, o componente de análise conceitual `TokenProcessor` foi implementado como a classe `TextParser`, a qual implementa a interface extensível `ITextParser` (promovendo desacoplamento e facilitando a substituição de regras sem alterar as classes consumidoras, em conformidade com o Princípio Aberto/Fechado - OCP).

Suposições de Projeto

Para o correto funcionamento do sistema e delimitação do escopo técnico na Fase 2, foram estabelecidas as seguintes suposições de projeto:

1. **Disponibilidade de SoundFonts**: O motor de síntese FluidSynth exige um arquivo de SoundFont (`.sf2` ou `.sf3`) General MIDI para gerar áudio analógico. Assume-se que o ambiente operacional possui caminhos comuns de bibliotecas de áudio populados (como `/usr/share/sounds/sf2/FluidR3_GM.sf2` no Linux). Como contingência, o sistema provê um diálogo interativo de seleção de arquivos (`QFileDialog`) caso nenhuma SoundFont padrão seja encontrada automaticamente, permitindo ao usuário indicar o caminho do arquivo manualmente.
2. **Reserva do Canal MIDI 9**: Por especificação do protocolo General MIDI, o canal 10 (indexado como 9) é estritamente dedicado a instrumentos de percussão. O sistema desconsidera o canal 9 para evitar problemas de compatibilidade com a sintetização de instrumentos diferentes. Assim, como MIDI 1.0 possui 16 canais, ficamos com 15 restantes, o que é mais do que o suficiente para o escopo desta aplicação. Além disso, não existe uma limitação de 15 'linhas', pois continuamos 're'mapeando linhas acima de 15 de volta para os canais iniciais. $n\%15$, n sendo número de linhas. No entanto, isso significa que mudanças de instrumento e volume em qualquer linha remapeada ao canal serão refletidas em todas as linhas que compartilham aquele canal.
3. **Restrições de BPM**: As instruções de aceleração (`>`) e desaceleração (`<`) possuem escopo global, como diz a especificação. No entanto, restringimos os valores máximo e mínimo para 1200 e 40, respecivamente, pois foi o valor que uma pesquisa rápida retornou como limites aceitáveis de BPM.

Descrição do Procedimento de Teste

A validação de corretude do sistema e do motor de áudio assíncrono abrangeu tanto testes unitários e arquivos de testes estruturados quanto testes de verificação manual do comportamento da interface e concorrência:

Casos de Teste Automatizados

Testes Unitários Automatizados: Implementados no arquivo `tests/test_backend.cpp` com o framework `doctest`, estes testes rodam programaticamente e de forma isolada das dependências físicas do disco. Eles utilizam sequências textuais equivalentes definidas diretamente em memória para validar com precisão matemática os estados internos e o enfileiramento de eventos.

Os testes automatizados cobrem a corretude das seguintes regras e componentes do sistema:

- **Configurações e Limites das Vozes**: Garante que os atributos iniciais das vozes (soprano, contralto, tenor, baixo) correspondam aos padrões definidos e que o mapeamento de canais MIDI respeite a reserva do canal 9 (percussão), roteando as vozes corretas e ciclando adequadamente.
- **Mapeamento de Notas e Frequências**: Valida a conversão de caracteres de notas (incluindo alterações bemóis como `Eb`, `Ab`, `Mb`) para a escala de pitch MIDI, assegurando que oitavas fora do limite 0-9 sejam tratadas por meio de clamping.
- **Processamento Lexical e Eventos**: Valida a extração de eventos como notas de repouso (pausa), dinâmicas de volume (espaço duplo), modificações de oitavas (`?`, `.`, `V`), comandos de instrumentos (dígitos, vogais, caracteres `!`, `;`, `,`) e repetição de notas com consoantes.
- **Polifonia e Sincronismo**: Assegura que o parser lidará corretamente com múltiplas linhas e que a sintaxe de atraso `[n]` gera os tempos corretos de silêncio iniciais para cada voz.
- **Modificadores Globais**: Testa a aceleração e desaceleração de BPM (`>` e `<`), garantindo o despacho correto do evento de andamento global sincronizado.

Verificação Manual de Concorrência e Estabilidade

Casos de Teste Estruturados (arquivos `.txt`): O diretório `tests/` concentra arquivos de texto simples contendo composições de teste padronizadas (como `fuga_bpm.txt`, `fuga_oitavas.txt`, etc.). Esses arquivos são ignorados pela execução automatizada e servem como massa de teste estruturada para validação visual e auditiva manual, sendo carregados e reproduzidos diretamente através da interface gráfica (GUI).

- **Fuga BWV 847** e **Fuga BWV 578**: Validação do processamento polifônico complexo, garantindo que múltiplas vozes com atrasos de entrada `[n]` rodem concorrentemente sem falhas de sincronia.
- **Fuga BPM**: Testa o processamento dos modificadores globais de andamento `>` e `<`, garantindo que o BPM do sequenciador seja acelerado e desacelerado de forma homogênea para todas as trilhas ativas.
- **Fuga Instrumentos**: Valida se modificadores como dígitos e caracteres especiais `!`, `;`, `,`, vogais) resultam nas corretas trocas de canais de instrumentos do sintetizador FluidSynth.
- **Fuga Pausas**: Valida o comportamento de silenciamento temporário introduzido por caracteres minúsculos `a`-`h` e outras consoantes.
- **Fuga Oitavas**: Confirma que os comandos `?`, `V` e `.` alteram as oitavas locais nos limites definidos (0 a 9) e que o estouro superior reinicia à oitava de partida.
- **Fuga Repetidas**: Valida a semântica de repetição de nota ao encontrar consoantes ou caracteres não mapeados logo após uma nota válida.